

❖ **Multi-threaded Programming:**

- Multithreading models
- Thread Libraries
- Threading issues.

❖ **Process Scheduling:**

- Basic concepts
- Scheduling Criteria;
- Scheduling Algorithms;
- Multiple-processor scheduling;
- Thread scheduling.

Unit- 2

MULTITHREADED PROGRAMMING

- A thread is a basic unit of CPU utilization.
 - It consists of
 - thread ID
 - PC
 - register-set and
 - stack.
 - It shares with other threads belonging to the same process its code-section & data-section.
 - A traditional (or heavy weight) process has a single thread of control.
 - If a process has multiple threads of control, it can perform more than one task at a time.
- such a process is called **multithreaded process**

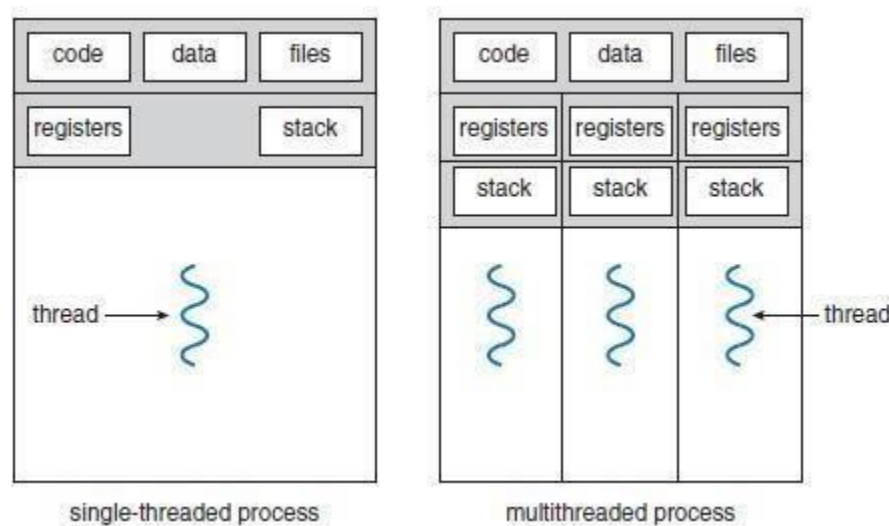


Fig: Single-threaded and multithreaded processes

Motivation for Multithreaded Programming

1. The software-packages that run on modern PCs are multithreaded. An application is implemented as a separate process with several threads of control. For ex: A word processor may have
 - first thread for displaying graphics
 - second thread for responding to keystrokes and
 - Third thread for performing grammar checking.

2. In some situations, a single application may be required to perform several similar tasks. For ex: A web-server may create a separate thread for each client requests. This allows the server to service several concurrent requests.
3. RPC servers are multithreaded.
 - When a server receives a message, it services the message using separate concurrent threads.
4. Most OS kernels are multithreaded;
 - Several threads operate in kernel, and each thread performs a specific task, such as managing devices or interrupt handling.

Benefits of Multithreaded Programming

- **Responsiveness** A program may be allowed to continue running even if part of it is blocked. Thus, increasing responsiveness to the user.
- **Resource Sharing** By default, threads share the memory (and resources) of the process to which they belong. Thus, an application is allowed to have several different threads of activity within the same address-space.
- **Economy** Allocating memory and resources for process-creation is costly. Thus, it is more economical to create and context-switch threads.
- **Utilization of Multiprocessor Architectures** In a multiprocessor architecture, threads may be running in parallel on different processors. Thus, parallelism will be increased.

MULTITHREADING MODELS

- Support for threads may be provided at either
 1. The user level, for **user threads** or
 2. By the kernel, for **kernel threads**.
- User-threads are supported above the kernel and are managed without kernel support. Kernel-threads are supported and managed directly by the OS.
- Three ways of establishing relationship between user-threads & kernel-threads:
 1. Many-to-one model
 2. One-to-one model and
 3. Many-to-many model.

Many-to-One Model

- Many user-level threads are mapped to one kernel thread.

Advantages:

- Thread management is done by the thread library in user space, so it is efficient.

Disadvantages:

- The entire process will block if a thread makes a blocking system-call.
- Multiple threads are unable to run in parallel on multiprocessors.

- For example:

- Solaris green threads
- GNU portable threads.

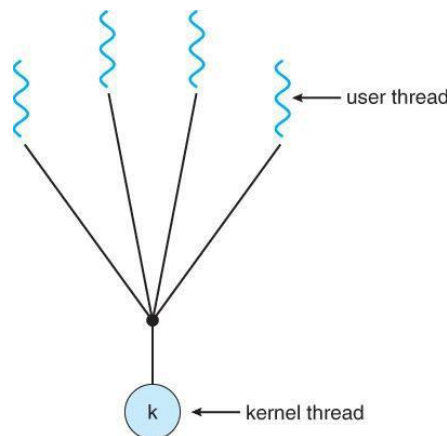


Fig: Many-to-one model

One-to-One Model

- Each user thread is mapped to a kernel thread.

Advantages:

- It provides more concurrency by allowing another thread to run when a thread makes a blocking system-call.
- Multiple threads can run in parallel on multiprocessors.

Disadvantage:

- Creating a user thread requires creating the corresponding kernel thread.

- For example:

- Windows NT/XP/2000, Linux

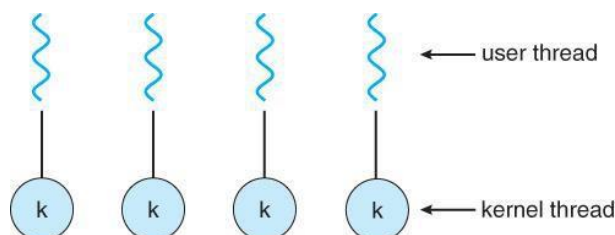


Fig: one-to-one model

Many-to-Many Model

- Many user-level threads are multiplexed to a smaller number of kernel threads.

Advantages:

- Developers can create as many user threads as necessary
- The kernel threads can run in parallel on a multiprocessor.
- When a thread performs a blocking system-call, kernel can schedule another thread for execution.

Two Level Model

- A variation on the many-to-many model is the two level-model
- Similar to M:N, except that it allows a user thread to be bound to kernel thread.
- for example:
 - HP-UX
 - Tru64 UNIX

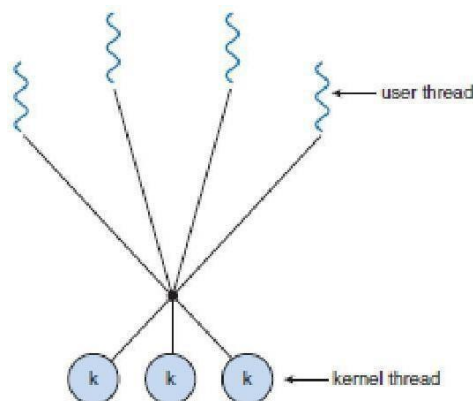


Fig: Many-to-many model

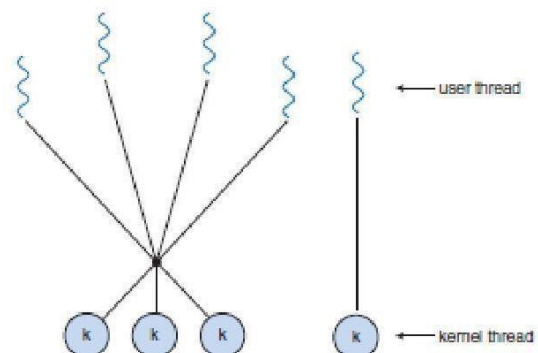


Fig: Two-level model

Thread Libraries

- It provides the programmer with an API for the creation and management of threads.

- Two ways of implementation:

1. First Approach:

Provides a library entirely in user space with no kernel support. All code and data structures for the library exist in the user space.

2. Second Approach

Provides a library entirely in user space with no kernel support. All code and data structures for the library exist in the user space.

Three main thread libraries:

1. POSIX threads
2. Win32 and
3. Java.

Pthreads

- This is a POSIX standard API for thread creation and synchronization.
- This is a specification for thread-behavior, not an implementation.
- OS designers may implement the specification in any way they wish.
- Commonly used in: UNIX and Solaris.

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }

    /* get the default attributes */
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}
```

```

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}

```

Win32 threads

- Implements the one-to-one mapping
- Each thread contains
 - A threadid
 - Registerset
 - Separate user and kernel stacks
 - Private data storage area
- The register set, stacks, and private storage area are known as the context of the threads. The primary data structures of a thread include:
 - ETHREAD (executive threadblock)
 - KTHREAD (kernel threadblock)
 - TEB (thread environmentblock)

Java Threads

- Threads are the basic model of program-execution in
 - Java program and
 - Java language.
- The API provides a rich set of features for the creation and management of threads.
- All Java programs comprise at least a single thread of control.
- Two techniques for creating threads:
 1. Create a new class that is derived from the Thread class and override its run() method.
 2. Define a class that implements the Runnable interface. The Runnable interface is defined as follows:

```

public interface Runnable
{
    public abstract void run();
}

```

THREADING ISSUES

fork() and exec() System-calls

- fork() is used to create a separate, duplicate process.
- If one thread in a program calls fork(), then
 1. Some systems duplicate all threads and
 2. Other systems duplicate only the thread that invoked the fork().
- If a thread invokes the exec(), the program specified in the parameter to exec() will replace the entire process including all threads.

Thread Cancellation

- This is the task of terminating a thread before it has completed.
- Target thread is the thread that is to be cancelled
- Thread cancellation occurs in two different cases:
 1. **Asynchronous cancellation**: One thread immediately terminates the target thread.
 2. **Deferred cancellation**: The target thread periodically checks whether it should be terminated.

Signal Handling

- In UNIX, a signal is used to notify a process that a particular event has occurred.
- All signals follow this pattern:
 1. A signal is generated by the occurrence of a certain event.
 2. A generated signal is delivered to a process.
 3. Once delivered, the signal must be handled.
- A signal handler is used to process signals.
- A signal may be received either synchronously or asynchronously, depending on the source.
 1. **Synchronous signals**
 - Delivered to the same process that performed the operation causing the signal.
 - E.g. illegal memory access and division by 0.
 2. **Asynchronous signals**
 - Generated by an event external to a running process.
 - E.g. user terminating a process with specific keystrokes <ctrl><c>.

- Every signal can be handled by one of two possible handlers:

1. A Default SignalHandler

- Run by the kernel when handling the signal.

2. A User-defined SignalHandler

- Overrides the default signal handler.

- In **single-threaded programs**, delivering signals is simple (since signals are always delivered to a process).
- In **multithreaded programs**, delivering signals is more complex. Then, the following options exist:
 1. Deliver the signal to the thread to which the signal applies.
 2. Deliver the signal to every thread in process
 3. Deliver the signal to certain threads in the process.
 4. Assign a specific thread to receive all signals for the process.

THREAD POOLS

- The basic idea is to
 - create a no. of threads at process-startup and
 - place the threads into a pool (where they sit and wait for work).
- Procedure:
 1. When a server receives a request, it awakens a thread from the pool.
 2. If any thread is available, the request is passed to it for service.
 3. Once the service is completed, the thread returns to the pool.
- Advantages:
 - Servicing a request with an existing thread is usually faster than waiting to create a thread.
 - The pool limits the no. of threads that exist at any one point.
- No. of threads in the pool can be based on factors such as
 - no. of CPUs
 - amount of memory and
 - expected no. of concurrent client-requests.

THREAD SPECIFIC DATA

- Threads belonging to a process share the data of the process.
- this sharing of data provides one of the benefits of multithreaded programming.
- In some circumstances, each thread might need its own copy of certain data. We will call such data **thread-specific data**.
- For example, in a transaction-processing system, we might service each transaction in a separate thread.

- Furthermore, each transaction may be assigned a unique identifier. To associate each thread with its unique identifier, we could use thread-specific data.

SCHEDULER ACTIVATIONS

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application.
- Scheduler activations provide **upcalls** a communication mechanism from the kernel to the thread library
- This communication allows an application to maintain the correct number kernel threads
- One scheme for communication between the user-thread library and the kernel is known as **scheduler activation**.

PROCESS SCHEDULING

Basic Concepts

- In a single-processor system,
 - Only one process may run at a time.
 - Other processes must wait until the CPU is rescheduled.
- Objective of multiprogramming:
 - To have some process running at all times, in order to maximize CPU utilization.

CPU-I/O Burst Cycle

- Process execution consists of a cycle of
 - CPU execution and
 - I/O wait
- Process execution begins with a CPU burst, followed by an I/O burst, then another CPU burst, etc...
- Finally, a CPU burst ends with a request to terminate execution.
- An I/O-bound program typically has many short CPU bursts.
- A CPU-bound program might have a few long CPU bursts.

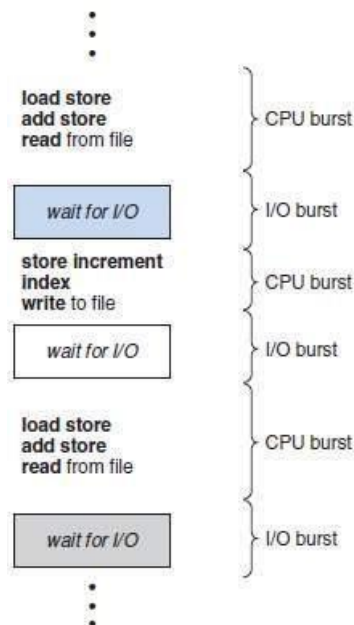


Fig Alternating sequence of CPU and I/O bursts

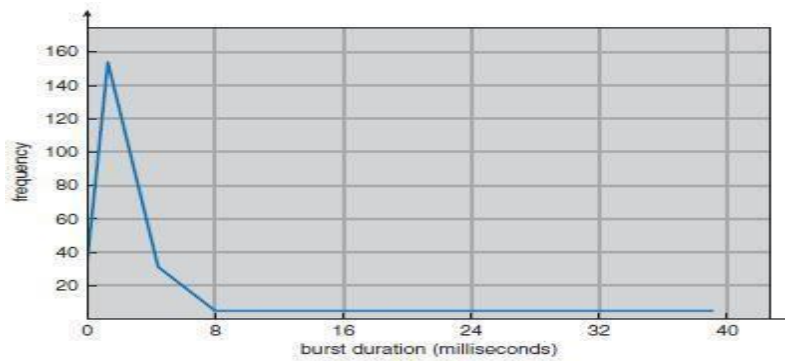


Fig: Histogram of CPU-burst durations

CPU Scheduler

- This scheduler
 - selects a waiting-process from the ready-queue and
 - allocates CPU to the waiting-process.
- The ready-queue could be a FIFO, priority queue, tree and list.
- The records in the queues are generally process control blocks (PCBs) of the processes.

CPU Scheduling

- Four situations under which CPU scheduling decisions take place:
 1. When a process switches from the running state to the waiting state. For ex; I/O request.
 2. When a process switches from the running state to the ready state. For ex: when an interrupt occurs.
 3. When a process switches from the waiting state to the ready state. For ex: completion of I/O.
 4. When a process terminates.
- Scheduling under 1 and 4 is non- preemptive. Scheduling under 2 and 3 is preemptive.

Non Preemptive Scheduling

- Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either
 - by terminating or
 - by switching to the waiting state.

Preemptive Scheduling

- This is driven by the idea of prioritized computation.
- Processes that are runnable may be temporarily suspended
- Disadvantages:
 1. Incurs a cost associated with access to shared-data.
 2. Affects the design of the OS kernel.

Dispatcher

- It gives control of the CPU to the process selected by the short-term scheduler.
- The function involves:
 1. Switching context
 2. Switching to user mode
 3. Jumping to the proper location in the user program to restart that program
- It should be as fast as possible, since it is invoked during every process switch.
- **Dispatch latency** means the time taken by the dispatcher to
 - stop one process and
 - start another running.

SCHEDULING CRITERIA:

In choosing which algorithm to use in a particular situation, depends upon the properties of the various algorithms. Many criteria have been suggested for comparing CPU-scheduling algorithms. The criteria include the following:

1. **CPU utilization:** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).
2. **Throughput:** If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput. For long processes, this rate may be one process per hour; for short transactions, it may be ten processes per second.
3. **Turnaround time.** This is the important criterion which tells how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the turnaround time. Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.
4. **Waiting time:** The CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/O, it affects only the amount of time that a process spends waiting in the ready queue. Waiting time is the sum of the periods spent waiting in the ready queue.
5. **Response time:** In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first response is produced. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response. The turnaround time is generally limited by the speed of the output device.

SCHEDULING ALGORITHMS

- CPU scheduling deals with the problem of deciding which of the processes in the ready-queue is to be allocated the CPU.
- Following are some scheduling algorithms:
 1. FCFS scheduling (First Come First Served)
 2. Round Robin scheduling
 3. SJF scheduling (Shortest Job First)
 4. SRT scheduling
 5. Priority scheduling
 6. Multilevel Queue scheduling
 7. Multilevel Feedback Queue scheduling

FCFS Scheduling

- The process that requests the CPU first is allocated the CPU first.
- The implementation is easily done using a FIFO queue.
- Procedure:
 1. When a process enters the ready-queue, its PCB is linked onto the tail of the queue.
 2. When the CPU is free, the CPU is allocated to the process at the queue's head.
 3. The running process is then removed from the queue.
- Advantage:
 1. Code is simple to write & understand.
- Disadvantages:
 1. **Convoy effect:** All other processes wait for one big process to get off the CPU.
 2. Non-preemptive (a process keeps the CPU until it releases it).
 3. Not good for time-sharing systems.
 4. The average waiting time is generally not minimal.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Example: Suppose that the processes arrive in the order P_1, P_2, P_3 .
- The Gantt Chart for the schedule is as follows:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
 Average waiting time: $(0 + 24 + 27)/3 = 17\text{ms}$

- Suppose that the processes arrive in the order P_2, P_3, P_1 .

- The Gantt chart for the schedule is as follows:



- Waiting time for P₁ = 6; P₂ = 0; P₃ = 3
- Average waiting time: $(6 + 0 + 3)/3 = 3\text{ms}$

SJF Scheduling

- The CPU is assigned to the process that has the smallest next CPU burst.
- If two processes have the same length CPU burst, FCFS scheduling is used to break the tie.
- For long-term scheduling in a batch system, we can use the process time limit specified by the user, as the 'length'.
- SJF can't be implemented at the level of short-term scheduling, because there is no way to know the length of the next CPU burst.
- Advantage:
 - The SJF is optimal, i.e. it gives the minimum average waiting time for a given set of processes.
- Disadvantage:
 - Determining the length of the next CPU burst.
- SJF algorithm may be either 1) non-preemptive or 2) preemptive.
 - 1. Non preemptive SJF**
 - The current process is allowed to finish its CPU burst.
 - 2. Preemptive SJF**
 - If the new process has a shorter next CPU burst than what is left of the executing process, that process is preempted. It is also known as **SRTF** scheduling (Shortest-Remaining-Time-First).
- Example (for **non-preemptive SJF**): Consider the following set of processes, with the length of the CPU-burst time given in milliseconds.

Process	Burst Time
P ₁	6
P ₂	8
P ₃	7
P ₄	3

- For non-preemptive SJF, the Gantt Chart is as follows:



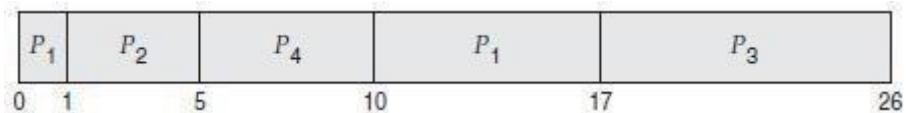
- Waiting time for $P_1 = 3$; $P_2 = 16$; $P_3 = 9$; $P_4 = 0$ Average waiting time: $(3 + 16 + 9 + 0)/4 = 7$

preemptive SJF/SRTF: Consider the following set of processes, with the length

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

of the CPU- burst time given in milliseconds.

- For preemptive SJF, the Gantt Chart is as follows:



- The average waiting time is $((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3))/4 = 26/4 = 6.5$.

Priority Scheduling

- A priority is associated with each process.
- The CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS order.
- Priorities can be defined either internally or externally.
 1. **Internally-defined** priorities.
 - Use some measurable quantity to compute the priority of a process.
 - For example: time limits, memory requirements, no. of open files.
 2. **Externally-defined** priorities.
 - Set by criteria that are external to the OS. For example:
 - importance of the process, political factors
- Priority scheduling can be either preemptive or non-preemptive.
 1. **Preemptive**

The CPU is preempted if the priority of the newly arrived process is higher than the priority of the currently running process.
 2. **Non Preemptive**

The new process is put at the head of the ready-queue
- Advantage:
 - Higher priority processes can be executed first.
- Disadvantage:
 - Indefinite blocking, where low-priority processes are left waiting indefinitely for CPU. Solution: **Aging** is a technique of increasing priority of processes that wait in system for a long time.

- Example: Consider the following set of processes, assumed to have arrived at time 0, in the order P₁, P₂, ..., P₅, with the length of the CPU-burst time given in milliseconds.

Process	Burst Time	Priority
P ₁	10	3
P ₂	1	1
P ₃	2	4
P ₄	1	5
P ₅	5	2

- The Gantt chart for the schedule is as follows:



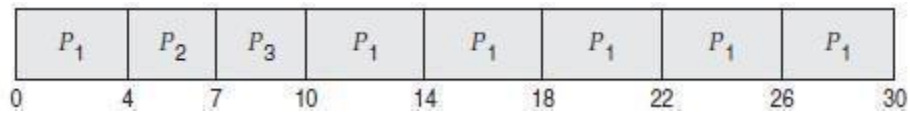
- The average waiting time is 8.2 milliseconds.

Round Robin Scheduling

- Designed especially for timesharing systems.
- It is similar to FCFS scheduling, but with preemption.
- A small unit of time is called a **time quantum** (or *timeslice*).
- Time quantum ranges from 10 to 100 ms.
- The ready-queue is treated as a **circular queue**.
- The CPU scheduler
 - goes around the ready-queue and
 - allocates the CPU to each process for a time interval of up to 1 time quantum.
- To implement:
 - The ready-queue is kept as a FIFO queue of processes
- CPU scheduler
 1. Picks the first process from the ready-queue.
 2. Sets a timer to interrupt after 1 time quantum and
 3. Dispatches the process.
- One of two things will then happen.
 1. The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily.
 2. If the CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the OS. The process will be put at the tail of the ready-queue.
- Advantage:
 - Higher average turnaround than SJF.
- Disadvantage:
 - Better response time than SJF.
- Example: Consider the following set of processes that arrive at time 0, with the length of the CPU-burst time given in milliseconds.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- The Gantt chart for the schedule is as follows:



- The average waiting time is $17/3 = 5.66$ milliseconds.
- The RR scheduling algorithm is preemptive.
 - No process is allocated the CPU for more than 1 time quantum in a row.
 - If a process' CPU burst exceeds 1 time quantum, that process is preempted and is put back in the ready-queue.
- The performance of algorithm depends heavily on the size of the time quantum.
 - If time quantum=very large, RR policy is the same as the FCFS policy.
 - If time quantum=very small, RR approach appears to the users as though each of n processes has its own processor running at $1/n$ the speed of the real processor.
- In software, we need to consider the effect of context switching on the performance of RR scheduling
 - Larger the time quantum for a specific process time, less time is spent on context switching.
 - The smaller the time quantum, more overhead is added for the purpose of context-switching.

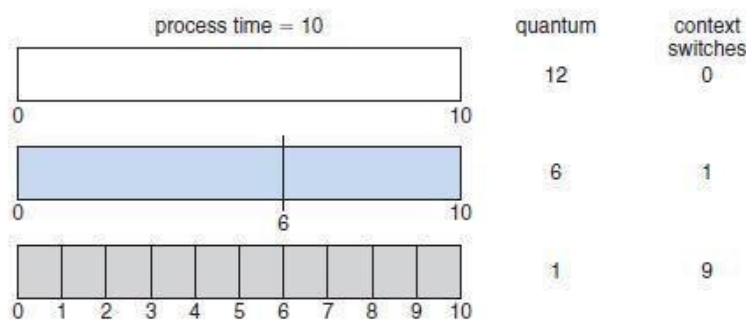


Fig: How a smaller time quantum increases context switches

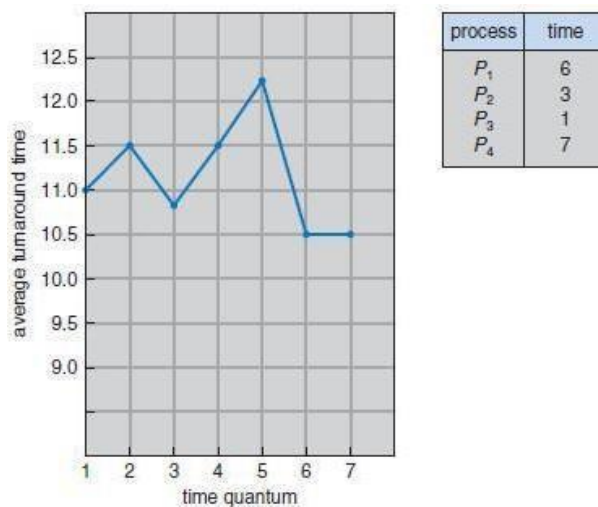


Fig: How turnaround time varies with the time quantum

Multilevel Queue Scheduling

- Useful for situations in which processes are easily classified into different groups.
 - For example, a common division is made between
 - foreground (or interactive) processes and
 - background (or batch) processes.
 - The ready-queue is partitioned into several separate queues (Figure 2.19).
 - The processes are permanently assigned to one queue based on some property like
 - memory size
 - process priority or
 - process type.
 - Each queue has its own scheduling algorithm.
- For example, separate queues might be used for foreground and background processes.

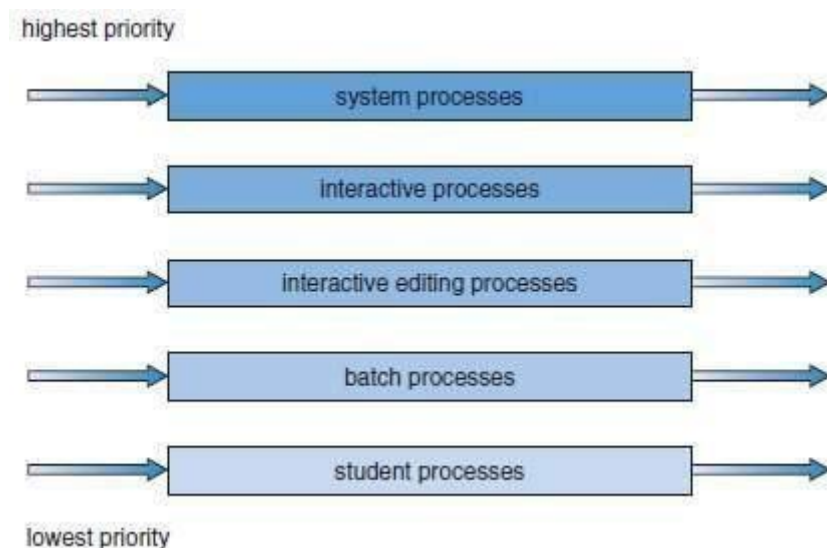


Fig Multilevel queue scheduling

- There must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling.
- For example, the foreground queue may have absolute priority over the background queue.
- **Time slice:** each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR 20% to background in FCFS

Multilevel Feedback Queue Scheduling

- A process may move between queues
- The basic idea: Separate processes according to the features of their CPU bursts. Forexample
 1. If a process uses too much CPU time, it will be moved to a lower-priority queue. This scheme leaves I/O-bound and interactive processes in the higher-priority queues.
 2. If a process waits too long in a lower-priority queue, it may be moved to a higher-priority queue This form of aging prevents starvation.

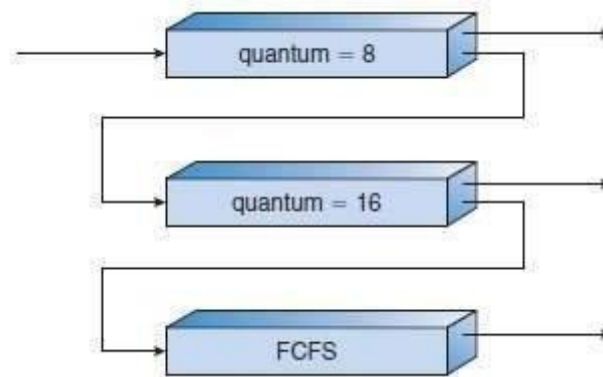


Figure 2.20 Multilevel feedback queues

In general, a multilevel feedback queue scheduler is defined by the following parameters:

1. The number of queues.
2. The scheduling algorithm for each queue.
3. The method used to determine when to upgrade a process to a higher priority queue.
4. The method used to determine when to demote a process to a lower priority queue.
5. The method used to determine which queue a process will enter when that process needs service

MULTIPLE PROCESSOR SCHEDULING

- If multiple CPUs are available, the scheduling problem becomes more complex.
- Two approaches:

Asymmetric Multiprocessing

The basic idea is:

- A master server is a single processor responsible for all scheduling decisions, I/O processing and other system activities.
- The other processors execute only user code.
- Advantage: This is simple because only one processor accesses the system data structures, reducing the need for data sharing.

Symmetric Multiprocessing

The basic idea is:

- Each processor is self-scheduling.
- To do scheduling, the scheduler for each processor
- Examines the ready-queue and
- Selects a process to execute.

Restriction: We must ensure that two processors do not choose the same process and that processes are not lost from the queue.

Processor Affinity

- In SMP systems,
 1. Migration of processes from one processor to another are avoided and
 2. Instead processes are kept running on same processor. This is known as processor affinity.

- Two forms:

1. Soft Affinity

- When an OS try to keep a process on one processor because of policy, but cannot guarantee it will happen.
- It is possible for a process to migrate between processors.

2. Hard Affinity

- When an OS have the ability to allow a process to specify that it is not to migrate to other processors. Eg: Solaris OS

Load Balancing

- This attempts to keep the workload evenly distributed across all processors in an SMP system.
- Two approaches:

1. Push Migration

A specific task periodically checks the load on each processor and if it finds an imbalance, it evenly distributes the load to idle processors.

2. Pull Migration

An idle processor pulls a waiting task from a busy processor.

Symmetric Multithreading

- The basic idea:
 1. Create multiple logical processors on the same physical processor.
 2. Present a view of several logical processors to the OS.
- Each logical processor has its own architecture state, which includes general-purpose and machine-state registers.
- Each logical processor is responsible for its own interrupt handling.
- SMT is a feature provided in hardware, not software.

THREAD SCHEDULING

- On OSs, it is kernel-level threads but not processes that are being scheduled by the OS.
- User-level threads are managed by a thread library, and the kernel is unaware of them.
- To run on a CPU, user-level threads must be mapped to an associated kernel-level thread.

Contention Scope

- Two approaches:
 1. *Process-Contention scope*
 - On systems implementing the many-to-one and many-to-many models, the thread library schedules user-level threads to run on an available LWP.
 - Competition for the CPU takes place among threads belonging to the same process.
 2. *System-Contentionscope*
 - The process of deciding which kernel thread to schedule on the CPU.
 - Competition for the CPU takes place among all threads in the system.
 - Systems using the one-to-one model schedule threads using only SCS.

Pthread Scheduling

- Pthread API that allows specifying either PCS or SCS during thread creation.
- Pthreads identifies the following contention scope values:
 1. PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling.
 2. PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling.
- Pthread IPC provides following two functions for getting and setting the contention scope policy:
 1. pthread_attr_setscope(pthread_attr_t *attr, int scope)
 2. pthread_attr_getscope(pthread_attr_t *attr, int *scope)

PROCESS SYNCHRONIZATION

- A cooperating process is one that can affect or be affected by other processes executing in the system. Cooperating processes can either directly share a logical address space (that is, both code and data) or be allowed to share data only through files or messages.
- Concurrent access to shared data may result in data inconsistency. To maintain data consistency, various mechanisms are required to ensure the orderly execution of cooperating processes that share a logical address space.

Producer- Consumer Problem

- A Producer process produces information that is consumed by consumer process.
- To allow producer and consumer process to run concurrently, A Bounded Buffer can be used where the items are filled in a buffer by the producer and emptied by the consumer.
- The original solution allowed at most **BUFFER_SIZE - 1** item in the buffer at the same time. To overcome this deficiency, an integer variable **counter**, initialized to 0 is added.
- **counter** is incremented every time when a new item is added to the buffer and is decremented every time when one item is removed from the buffer.

The code for the **producer process** can be modified as follows:

```
while (true) {
    /* produce an item and put in nextProduced*/ while
    (counter == BUFFER_SIZE)
        ; // do nothing
    buffer [in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

The code for the **consumer process** can be modified as follows:

```
while (true){
    while (counter ==0)
        ; // do nothing
    nextConsumed =buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* consume the item in nextConsumed */
}
```

- **Race Condition**

When the producer and consumer routines shown above are correct separately, they may not function correctly when executed concurrently.

- **Illustration:**

Suppose that the value of the variable **counter** is currently 5 and that the producer and consumer processes execute the statements "counter++" and "counter--" concurrently. The value of the variable counter may be 4, 5, or 6 but the only correct result is counter == 5, which is generated correctly if the producer and consumer execute separately.

The value of **counter** may be incorrect as shown below:

The statement counter++ could be implemented as

```
register1= counter
register1 = register1 + 1
counter =register1
```

The statement counter-- could be implemented as

```
register2 =counter
register2 = register2 - 1
count = register2
```

The concurrent execution of "counter++" and "counter--" is equivalent to a sequential execution in which the lower-level statements presented previously are interleaved in some arbitrary order. One such interleaving is

Consider this execution interleaving with "count = 5" initially:

S0: producer execute register1=counter	{register1 = 5}
S1: producer execute register1 = register1+1	{register1 = 6}
S2: consumer execute register2=counter	{register2 = 5}
S3: consumer execute register2 = register2-1	{register2 = 4}
S4: producer execute counter=register1	{count =6}
S5: consumer execute counter=register2	{count =4}

- **Note:** It is arrived at the incorrect state "counter == 4", indicating that four buffers are full, when, in fact, five buffers are full. If we reversed the order of the statements at T4 and T5, we would arrive at the incorrect state "counter==6".
- **Definition Race Condition:** A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **RaceCondition**.
- To guard against the race condition, ensure that only one process at a time can be manipulating the variable counter. To make such a guarantee, **the processes are synchronized** in some way.

The Critical Section Problems

- Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$.
- Each process has a segment of code, called a **critical section** in which the process may be changing common variables, updating a table, writing a file, and soon
- The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section. That is, no two processes are executing in their critical sections at the sametime.
- The critical-section problem is to design a protocol that the processes can use to cooperate.

The general structure of a typical process P_i is shown in below figure.

- Each process must request permission to enter its critical section. The section of code implementing this request is the **entry section**.
- The critical section may be followed by an **exit section**. The remaining code is the **remainder section**.

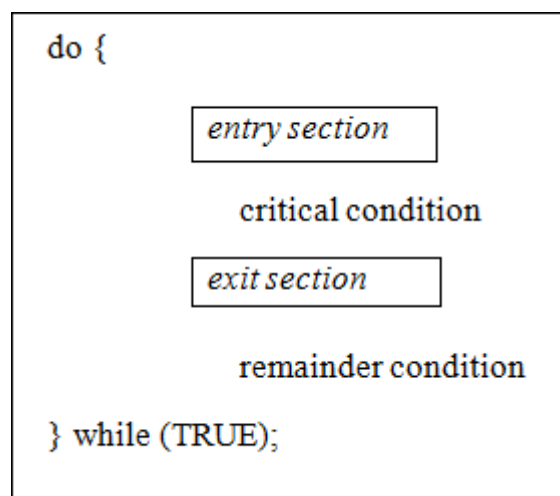


Figure: General structure of a typical process P_i

A solution to the critical-section problem must satisfy the following **three requirements**:

1. **Mutual exclusion**: If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress**: If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
3. **Bounded waiting**: There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

SEMAPHORE

- A semaphore is a synchronization tool is used solve various synchronization problem and can be implemented efficiently.
- Semaphore do not require busywaiting.
- A semaphore S is an integer variable that, is accessed only through two standard atomic operations: wait () and signal (). The wait () operation was originally termed P and signal() was called V.

Definition of wait ():

```
wait (S) {  
    while S <= 0  
        ; // no-op  
    S--;
```

Definition of signal ():

```
signal (S) {  
    S++;}
```

- All modifications to the integer value of the semaphore in the wait () and signal() operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Binary semaphore

- The value of a binary semaphore can range only between 0 and 1.
- Binary semaphores are known as mutex locks, as they are locks that provide mutual exclusion. Binary semaphores to deal with the critical-section problem for multiple processes. Then processes share a semaphore, mutex, initialized to 1

Each process P_i is organized as shown in below figure

```
do {
    wait (mutex);
        // Critical Section
    signal (mutex);
        // remainder section
} while (TRUE);
```

Figure: Mutual-exclusion implementation with semaphores

Counting semaphore

- The value of a counting semaphore can range over an unrestricted domain.
- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.
- The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a wait() operation on the semaphore. When a process releases a resource, it performs a signal() operation.
- When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

Implementation

- The main disadvantage of the semaphore definition requires busy waiting.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.
- This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is shared among many processes.

- Busy waiting wastes CPU cycles that some other process might be able to use productively. This type of semaphore is also called a **spinlock** because the process "spins" while waiting for the lock.

Semaphore implementation with no busy waiting

- The definition of the wait() and signal() semaphore operations is modified.
- When a process executes the wait () operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, the process can block itself. The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state. Then control is transferred to the CPU scheduler, which selects another process to execute.
- A process that is blocked, waiting on a semaphore S, should be restarted when some other process executes a signal() operation. The process is restarted by a wakeup() operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue.
- To implement semaphores under this definition, we define a semaphore as a "C" struct:

```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

- Each semaphore has an integer value and a list of processes list. When a process must wait on a semaphore, it is added to the list of processes. A signal() operation removes one process from the list of waiting processes and awakens that process.
- The wait() semaphore operation can now be defined as:

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S-
        >list; block();
    }
}
```

- The `signal()` semaphore operation can now be defined as

```

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P
        from S->list;
        wakeup(P);
    }
}

```

- The `block()` operation suspends the process that invokes it. The `wakeup(P)` operation resumes the execution of a blocked process P. These two operations are provided by the operating system as basic systemcalls.
- In this implementation semaphore values may be negative. If a semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.

Deadlocks and Starvation

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the execution of a `signal()` operation. When such a state is reached, these processes are said to be deadlocked.
- To illustrate this, consider a system consisting of two processes, P_0 and P_1 , each accessing two semaphores, S and Q, set to the value 1

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
signal(S);	signal(Q);
signal(Q);	signal(S);

- Suppose that P_0 executes wait (S) and then P_1 executes wait (Q). When P_0 executes wait (Q), it must wait until P_1 executes signal (Q). Similarly, when P_1 executes wait (S), it must wait until P_0 executes signal(S). Since these `signal()` operations cannot be executed, P_0 and P_1 are deadlocked.

- Another problem related to deadlocks is indefinite blocking or starvation: A situation in which processes wait indefinitely within the semaphore.
- Indefinite blocking may occur if we remove processes from the list associated with a semaphore in LIFO (last-in, first-out) order.

CLASSICAL PROBLEMS OF SYNCHRONIZATION

- Bounded-BufferProblem
- Readers and WritersProblem
- Dining-PhilosophersProblem

Bounded-Buffer Problem

- N buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value N.

```
while (true)
{
    // produce an item
    wait (empty);
    wait (mutex);
    // add the item to the buffer
    signal (mutex);
    signal (full);
}
```

The structure of the producer process:

```
while (true)
{
    wait (full);
    wait (mutex);
    // remove an item from buffer
    signal (mutex);
    signal (empty);
    // consume the removed item
}
```

The structure of the consumer process:

Readers-Writers Problem

- A data set is shared among a number of concurrent processes
- Readers – only read the data set; they do **not** perform any updates
- Writers – can both read and write.
- Problem – allow multiple readers to read at the same time. Only one single writer can access the shared data at the same time.
 - SharedData
 - Dataset
 - Semaphore **mutex** initialized to 1.
 - Semaphore **wrt** initialized to 1.
 - Integer **readcount** initialized to 0.

```
while (true)
{
    wait (wrt) ;
    // writing is performed
    signal (wrt) ;
}
```

The structure of a writer process

```
while (true)
{
    wait (mutex) ;
    readcount ++ ;
    if (readcount == 1)
        wait (wrt) ;
    signal (mutex)
    // reading is performed
    wait (mutex) ;
    readcount -- ;
    if (readcount == 0)
        signal (wrt) ;
    signal (mutex) ;
}
```

The structure of a reader process

Dining-Philosophers Problem

Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chopsticks.



A philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors). A philosopher may pick up only one chopstick at a time. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing the chopsticks. When she is finished eating, she puts down both chopsticks and starts thinking again.

It is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner.

Solution: One simple solution is to represent each chopstick with a semaphore. A philosopher tries to grab a chopstick by executing a `wait()` operation on that semaphore. She releases her chopsticks by executing the `signal()` operation on the appropriate semaphores. Thus, the shared data are

semaphore chopstick[5];

where all the elements of chopstick are initialized to 1. The structure of philosopher *i* is shown

```
while (true)
{
    wait ( chopstick[i] );
    wait ( chopstick[ (i + 1) % 5] );
    // eat
    signal ( chopstick[i] );
    signal ( chopstick[ (i + 1) % 5] );
    // think
}
```

Several possible remedies to the deadlock problem are replaced by:

- Allow at most four philosophers to be sitting simultaneously at the table.

- Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even numbered philosopher picks up her right chopstick and then her left chopstick.

Problems with Semaphores

Correct use of semaphore operations:

- `signal (mutex) ... wait (mutex)` : Replace `signal` with `wait` and vice-versa
 - `wait (mutex) ... wait(mutex)`
 - Omitting of `wait (mutex)` or `signal (mutex)` (or both)
-